



database.md — Orvio Data Architecture & Schema Specification

Version: 0.1 (MVP-ready)

Last updated: 2026-09-28

Owner: Engineering (Code X)

Applies to: Backend, Frontend, DevOps, QA, AI coding agents

0. How to Use This Document

This is the **database source of truth** for Orvio. It defines what data exists, why it exists, how it relates, constraints, ownership, lifecycle, and how to change it safely.

Chain of truth:

PRD → `architecture.md` → `api-contract.md` → `database.md` → Migrations → Code

Rules:

- Do not change schema without updating this doc and migrations.
- Do not bypass constraints or tenant isolation.
- AI agents must read this before implementing data-related features.

1. Core Principle

Treat the database as a **critical system boundary**.

This document defines:

- What data exists and why.
- Relationships and ownership.
- Required/optional fields, allowed values, constraints.
- Create/update/delete/archive rules.
- Query patterns and isolation.
- Safe change management.

Status labels:

- **Confirmed requirement:** From PRD/architecture.

- **Technical recommendation:** Engineering guidance.
- **Assumption:** Reasonable inference, may need validation.
- **OPEN DECISION:** Not yet decided; do not assume.

2. Database Technology

Engine: PostgreSQL 15+ (managed).

ORM: Prisma (type-safe queries, migrations).

Hosting: PaaS managed PostgreSQL (Render/Railway/Fly).

Connection: Connection pooling via Prisma; env-based connection strings.

Migrations: `prisma migrate (dev → staging → prod)`.

Local dev: Local PostgreSQL (Docker or native).

Test: Isolated DB per CI run (ephemeral).

Staging: Managed DB (staging env).

Production: Managed DB (prod env, daily backups).

PostgreSQL considerations:

- Use UUID for IDs (primary keys).
- Use TIMESTAMPTZ for timestamps (UTC).
- Use DECIMAL for money (minor units as integer also acceptable; see schema).
- Row-Level Security (RLS) deferred to Phase 2; enforce `org_id` in application layer (repository).

3. Database Architecture

3.1 Role in System

- **Single source of truth** for all tenant data (organizations, users, products, stock, sales, orders, payments).
- **Backend-only access;** no direct client access.
- **Transactional integrity** for stock movements, sales, payments.

3.2 Data Flow

```

flowchart LR
    Client[React Client] --> API[Fastify API]
    API --> DataAccess[Repository/Prisma]
    DataAccess --> DB[(PostgreSQL)]
    API --> Cache[(In-memory cache)]
    API --> Worker[Background Worker]
    Worker --> DB
    Worker --> External[Paystack/WhatsApp]
  
```

3.3 Boundaries

- **Read/write:** All via backend repository layer.
- **Transactions:** Wrap multi-step operations (sale → stock movement).
- **Caching:** In-memory config; TanStack Query on frontend.
- **Background jobs:** Process webhooks, retries, scheduled tasks.

4. Multi-Tenancy & Data Isolation

Tenant = Organization. Every row (except global config) scoped by `org_id`.

Isolation strategy (MVP):

- Application-enforced `WHERE org_id = ?` in repository base class.
- All queries include `org_id` from authenticated context.
- Tests verify no cross-org access.

Future: Row-Level Security (RLS) policies.

Security rule: Never trust client-provided `org_id`; derive from token/session.

5. Naming Conventions

- **Tables:** `snake_case`, plural (e.g., `products`, `sales_orders`).
- **Columns:** `snake_case` (e.g., `org_id`, `created_at`).
- **Indexes:** `table_column_idx` (e.g., `products_org_id_idx`).
- **FKs:** `table_id` (e.g., `org_id`, `branch_id`).
- **UUIDs:** Default for primary keys.

6. Conceptual Data Model (ERD)

```
erDiagram
    ORGANIZATION ||--o{ USER_MEMBERSHIP : includes
    ORGANIZATION ||--o{ BRANCH : has
    ORGANIZATION ||--o{ PRODUCT : owns
    ORGANIZATION ||--o{ SALES_ORDER : receives
    ORGANIZATION ||--o{ PAYMENT_INTENT : billed_as
    ORGANIZATION ||--o{ WHATSAPP_CHANNEL : configures
    ORGANIZATION ||--o{ SUBSCRIPTION : subscribed_as

    BRANCH ||--o{ STOCK_ITEM : holds
    BRANCH ||--o{ STOCK_MOVEMENT : occurs_at
    BRANCH ||--o{ SALES_ORDER : receives

    PRODUCT ||--o{ STOCK_ITEM : tracked_in
    PRODUCT ||--o{ STOCK_MOVEMENT : generates
```

```
PRODUCT ||--o{ SALES_ORDER_ITEM : sold_as

SALES_ORDER ||--o{ SALES_ORDER_ITEM : contains
SALES_ORDER ||--o{ PAYMENT_INTENT : paid_by

USER ||--o{ USER_MEMBERSHIP : belongs_to

WHATSAPP_CHANNEL ||--o{ WHATSAPP_ORDER : receives
```

7. Global Conventions

7.1 Common Columns

- `id` UUID PRIMARY KEY DEFAULT `gen_random_uuid()`
- `org_id` UUID NOT NULL (tenant scope)
- `created_at` TIMESTAMPTZ NOT NULL DEFAULT `NOW()`
- `updated_at` TIMESTAMPTZ NOT NULL DEFAULT `NOW()`
- `deleted_at` TIMESTAMPTZ NULL (soft delete)
- `created_by` UUID NULL (user who created)
- `updated_by` UUID NULL

7.2 Soft Delete

- All core tables use `deleted_at`.
- Queries filter `deleted_at IS NULL` by default.
- Hard delete after retention period (OPEN DECISION: 90 days).

7.3 Money

- Represent as **integer minor units** (e.g., NGN kobo).
- Column: `amount` BIGINT NOT NULL, `currency` CHAR(3) NOT NULL DEFAULT `'NGN'`.

7.4 Dates/Times

- All timestamps TIMESTAMPTZ, stored in UTC.
- API returns ISO 8601 (2026-09-28T22:30:00Z).

7.5 Enums

Use PostgreSQL ENUM or application-level enums (Prisma).

Examples:

- `user_status`: active, suspended, pending, deleted.
- `order_status`: draft, pending_payment, paid, fulfilled, refunded, cancelled.

- payment_status: pending, succeeded, failed, refunded.
- stock_movement_type: in, out, transfer, adjust.

8. Core Tables

8.1 organizations

Purpose: Tenant boundary; each business is an org.

Column	Type	Required	Nullable	Validation	Description
id	UUID	PK	No	—	Org ID
name	TEXT	Yes	No	1–120 chars	Business name
address	TEXT	No	Yes	—	Business address
timezone	TEXT	Yes	No	IANA tz	Default timezone
currency	CHAR(3)	Yes	No	ISO 4217	Default currency (NGN)
tax_settings	JSONB	No	Yes	—	Tax config (VAT rate, etc.)
status	ENUM	Yes	No	active, suspended, deleted	Org status
created_at	TIMESTAMPTZ	Yes	No	—	—
updated_at	TIMESTAMPTZ	Yes	No	—	—
deleted_at	TIMESTAMPTZ	No	Yes	—	Soft delete

Constraints:

- UNIQUE (name) per env (optional).
- Index on status.

Lifecycle:

- Create on registration.
- Suspend via admin.
- Soft delete on owner request.

8.2 branches

Purpose: Physical locations (stores, warehouses).

Column	Type	Required	Nullable	Validation	Description
id	UUID	PK	No	—	Branch ID

Column	Type	Required	Nullable	Validation	Description
org_id	UUID	Yes	No	FK → <u>orgs.id</u>	Org owner
name	TEXT	Yes	No	1–120 chars	Branch name
type	ENUM	Yes	No	store, warehouse	Branch type
address	TEXT	No	Yes	—	Address
status	ENUM	Yes	No	active, inactive, deleted	Status
created_at	TIMESTAMPTZ	Yes	No	—	—
updated_at	TIMESTAMPTZ	Yes	No	—	—
deleted_at	TIMESTAMPTZ	No	Yes	—	Soft delete

Constraints:

- FK org_id → organizations(id).
- Index on org_id, status.

Lifecycle:

- Create by Owner/Manager.
- Deactivate instead of delete when possible.

8.3 users

Purpose: Global user accounts (auth identities).

Column	Type	Required	Nullable	Validation	Description
id	UUID	PK	No	—	User ID
email	TEXT	Yes	No	Unique, email format	Email
phone	TEXT	No	Yes	E.164	Phone
password_hash	TEXT	Yes	No	—	Argon2/bcrypt hash
status	ENUM	Yes	No	active, suspended, deleted	Status
mfa_enabled	BOOL	Yes	No	Default false	MFA flag
created_at	TIMESTAMPTZ	Yes	No	—	—
updated_at	TIMESTAMPTZ	Yes	No	—	—
deleted_at	TIMESTAMPTZ	No	Yes	—	Soft delete

Constraints:

- UNIQUE (email).

- Index on email, status.

Lifecycle:

- Create on registration.
- Suspend for abuse.
- Soft delete on request.

8.4 user_memberships

Purpose: Link users to orgs with roles.

Column	Type	Required	Nullable	Validation	Description
id	UUID	PK	No	—	Membership ID
org_id	UUID	Yes	No	FK → <u>orgs.id</u>	Org
user_id	UUID	Yes	No	FK → <u>users.id</u>	User
role	ENUM	Yes	No	owner, manager, cashier, storekeeper, accountant	Role
status	ENUM	Yes	No	active, inactive, deleted	Status
invited_at	TIMESTAMPTZ	No	Yes	—	Invite timestamp
accepted_at	TIMESTAMPTZ	No	Yes	—	Accept timestamp
created_at	TIMESTAMPTZ	Yes	No	—	—
updated_at	TIMESTAMPTZ	Yes	No	—	—
deleted_at	TIMESTAMPTZ	No	Yes	—	Soft delete

Constraints:

- UNIQUE (org_id, user_id).
- FKs to organizations, users.
- Index on org_id, user_id.

Lifecycle:

- Create on org creation (owner).
- Invite flow sets invited_at; accept sets accepted_at.

8.5 products

Purpose: Catalog items sold/tracked.

Column	Type	Required	Nullable	Validation	Description
id	UUID	PK	No	—	Product ID
org_id	UUID	Yes	No	FK → <u>orgs.id</u>	Org owner
sku	TEXT	Yes	No	Unique per org, 1–60 chars	SKU
name	TEXT	Yes	No	1–200 chars	Name
description	TEXT	No	Yes	—	Description
cost_price	BIGINT	Yes	No	≥ 0	Cost (minor units)
sell_price	BIGINT	Yes	No	≥ 0	Sell price
tax_rate	DECIMAL(5,2)	No	Yes	0–100	Tax %
barcode	TEXT	No	Yes	—	Barcode
category	TEXT	No	Yes	—	Category
is_active	BOOL	Yes	No	Default true	Active flag
created_at	TIMESTAMPTZ	Yes	No	—	—
updated_at	TIMESTAMPTZ	Yes	No	—	—
deleted_at	TIMESTAMPTZ	No	Yes	—	Soft delete

Constraints:

- UNIQUE (org_id, sku).
- Index on org_id, sku, is_active.

Lifecycle:

- Create by Owner/Manager/Storekeeper.
- Deactivate instead of delete.

8.6 stock_items

Purpose: Stock on hand per product per branch.

Column	Type	Required	Nullable	Validation	Description
id	UUID	PK	No	—	StockItem ID
org_id	UUID	Yes	No	FK → <u>orgs.id</u>	Org
branch_id	UUID	Yes	No	FK → <u>branches.id</u>	Branch
product_id	UUID	Yes	No	FK → <u>products.id</u>	Product

Column	Type	Required	Nullable	Validation	Description
quantity_on_hand	BIGINT	Yes	No	≥ 0	Current qty
reserved_quantity	BIGINT	Yes	No	$\geq 0, \leq \text{on_hand}$	Reserved
created_at	TIMESTAMPTZ	Yes	No	—	—
updated_at	TIMESTAMPTZ	Yes	No	—	—
deleted_at	TIMESTAMPTZ	No	Yes	—	Soft delete

Constraints:

- UNIQUE (org_id, branch_id, product_id).
- Check reserved_quantity $\leq$ quantity_on_hand.
- Index on org_id, branch_id, product_id.

Lifecycle:

- Created on initial stock import.
- Updated by stock movements.

8.7 stock_movements

Purpose: Audit trail of all stock changes.

Column	Type	Required	Nullable	Validation	Description
id	UUID	PK	No	—	Movement ID
org_id	UUID	Yes	No	FK $\rightarrow$ <u>orgs.id</u>	Org
branch_id	UUID	Yes	No	FK $\rightarrow$ <u>branches.id</u>	Branch
product_id	UUID	Yes	No	FK $\rightarrow$ <u>products.id</u>	Product
type	ENUM	Yes	No	in, out, transfer, adjust	Type
quantity	BIGINT	Yes	No	$\neq 0$	Qty change (signed)
reason	TEXT	No	Yes	Enum: damage, loss, found, return, sale, receive, transfer	Reason
reference_type	TEXT	No	Yes	sales_order, purchase_order, transfer, adjustment	Ref type
reference_id	UUID	No	Yes	FK (polymorphic)	Ref ID
notes	TEXT	No	Yes	—	Notes
created_by	UUID	No	Yes	FK $\rightarrow$ <u>users.id</u>	Creator

Column	Type	Required	Nullable	Validation	Description
created_at	TIMESTAMPTZ	Yes	No	—	—

Constraints:

- FKs to orgs, branches, products.
- Index on org_id, branch_id, product_id, type, created_at.

Lifecycle:

- Created on sale, receive, adjust, transfer.
- Never deleted (audit).

8.8 sales_orders

Purpose: Sales transactions (POS, WhatsApp, web).

Column	Type	Required	Nullable	Validation	Description
id	UUID	PK	No	—	Order ID
org_id	UUID	Yes	No	FK → <u>orgs.id</u>	Org
branch_id	UUID	Yes	No	FK → <u>branches.id</u>	Branch
channel	ENUM	Yes	No	pos, whatsapp, web	Channel
status	ENUM	Yes	No	draft, pending_payment, paid, fulfilled, refunded, cancelled	Status
subtotal	BIGINT	Yes	No	≥ 0	Subtotal (minor)
tax	BIGINT	Yes	No	≥ 0	Tax amount
total	BIGINT	Yes	No	≥ 0	Total
payment_status	ENUM	Yes	No	pending, paid, failed, refunded	Payment status
customer_phone	TEXT	No	Yes	E.164	Customer phone
created_by	UUID	No	Yes	FK → <u>users.id</u>	Creator
created_at	TIMESTAMPTZ	Yes	No	—	—
updated_at	TIMESTAMPTZ	Yes	No	—	—
deleted_at	TIMESTAMPTZ	No	Yes	—	Soft delete

Constraints:

- Index on org_id, branch_id, channel, status, created_at.

Lifecycle:

- Create on POS checkout or WhatsApp order.
- Transition to paid on payment success.

8.9 sales_order_items

Purpose: Line items in a sales order.

Column	Type	Required	Nullable	Validation	Description
id	UUID	PK	No	—	Item ID
order_id	UUID	Yes	No	FK → sales_orders.id	Order
product_id	UUID	Yes	No	FK → <u>products.id</u>	Product
quantity	BIGINT	Yes	No	> 0	Qty
unit_price	BIGINT	Yes	No	≥ 0	Price at sale
discount	BIGINT	Yes	No	≥ 0	Discount
tax	BIGINT	Yes	No	≥ 0	Tax
created_at	TIMESTAMPTZ	Yes	No	—	—

Constraints:

- UNIQUE (order_id, product_id) (or allow multiples with separate rows).
- Index on order_id, product_id.

Lifecycle:

- Created with order.
- Never updated post-payment (audit).

8.10 payment_intents

Purpose: Track payment attempts (Paystack, etc.).

Column	Type	Required	Nullable	Validation	Description
id	UUID	PK	No	—	PaymentIntent ID
org_id	UUID	Yes	No	FK → <u>orgs.id</u>	Org
order_id	UUID	Yes	No	FK → sales_orders.id	Order
provider	TEXT	Yes	No	paystack, flutterwave, moniepoint	Provider

Column	Type	Required	Nullable	Validation	Description
provider_ref	TEXT	No	Yes	Unique	Provider reference
amount	BIGINT	Yes	No	≥ 0	Amount
currency	CHAR(3)	Yes	No	NGN	Currency
status	ENUM	Yes	No	pending, succeeded, failed, refunded	Status
metadata	JSONB	No	Yes	—	Extra data
created_at	TIMESTAMPTZ	Yes	No	—	—
updated_at	TIMESTAMPTZ	Yes	No	—	—

Constraints:

- UNIQUE (provider_ref).
- Index on org_id, order_id, status.

Lifecycle:

- Create on payment link generation.
- Update on webhook.

8.11 whatsapp_channels

Purpose: WhatsApp Business API config per org.

Column	Type	Required	Nullable	Validation	Description
id	UUID	PK	No	—	Channel ID
org_id	UUID	Yes	No	FK → <u>orgs.id</u>	Org
phone_number	TEXT	Yes	No	E.164	WA number
provider	TEXT	Yes	No	wati, twilio, prembly	BSP
status	ENUM	Yes	No	active, inactive, deleted	Status
config	JSONB	No	Yes	—	Secrets, template IDs
created_at	TIMESTAMPTZ	Yes	No	—	—
updated_at	TIMESTAMPTZ	Yes	No	—	—
deleted_at	TIMESTAMPTZ	No	Yes	—	Soft delete

Constraints:

- UNIQUE (org_id, phone_number).

- Index on org_id, status.

Lifecycle:

- Create by Owner/Manager.
- Deactivate when not in use.

8.12 whatsapp_orders

Purpose: Orders captured via WhatsApp.

Column	Type	Required	Nullable	Validation	Description
id	UUID	PK	No	—	WA Order ID
org_id	UUID	Yes	No	FK → <u>orgs.id</u>	Org
channel_id	UUID	Yes	No	FK → whatsapp_channels.id	Channel
customer_phone	TEXT	Yes	No	E.164	Customer
items_json	JSONB	Yes	No	—	Cart items
total	BIGINT	Yes	No	≥ 0	Total
status	ENUM	Yes	No	pending_payment, paid, fulfilled, cancelled	Status
payment_link_id	UUID	No	Yes	FK → payment_intents.id	Payment
created_at	TIMESTAMPTZ	Yes	No	—	—
updated_at	TIMESTAMPTZ	Yes	No	—	—
deleted_at	TIMESTAMPTZ	No	Yes	—	Soft delete

Constraints:

- Index on org_id, status, created_at.

Lifecycle:

- Create on WhatsApp cart submission.
- Transition to paid on webhook.

8.13 subscriptions

Purpose: SaaS billing per org.

Column	Type	Required	Nullable	Validation	Description
id	UUID	PK	No	—	Subscription ID

Column	Type	Required	Nullable	Validation	Description
org_id	UUID	Yes	No	FK → <u>orgs.id</u>	Org
plan	TEXT	Yes	No	free, starter, growth	Plan
status	ENUM	Yes	No	trial, active, past_due, canceled	Status
current_period_start	TIMESTAMPTZ	Yes	No	—	Period start
current_period_end	TIMESTAMPTZ	Yes	No	—	Period end
provider	TEXT	No	Yes	paystack, stripe	Provider
provider_customer_id	TEXT	No	Yes	—	Provider customer ID
created_at	TIMESTAMPTZ	Yes	No	—	—
updated_at	TIMESTAMPTZ	Yes	No	—	—
deleted_at	TIMESTAMPTZ	No	Yes	—	Soft delete

Constraints:

- UNIQUE (org_id).
- Index on org_id, status.

Lifecycle:

- Create on signup (free plan).
- Update on payment success/failure.

8.14 audit_logs

Purpose: Security/operations audit trail.

Column	Type	Required	Nullable	Validation	Description
id	UUID	PK	No	—	Log ID
org_id	UUID	Yes	No	FK → <u>orgs.id</u>	Org
actor_user_id	UUID	No	Yes	FK → <u>users.id</u>	Actor
action	TEXT	Yes	No	—	Action code
resource_type	TEXT	Yes	No	—	Resource type
resource_id	UUID	No	Yes	—	Resource ID
before_json	JSONB	No	Yes	—	Before state

Column	Type	Required	Nullable	Validation	Description
after_json	JSONB	No	Yes	—	After state
ip	TEXT	No	Yes	—	IP address
user_agent	TEXT	No	Yes	—	UA
created_at	TIMESTAMPTZ	Yes	No	—	—

Constraints:

- Index on org_id, actor_user_id, resource_type, created_at.

Lifecycle:

- Append-only; never updated/deleted (retention policy OPEN DECISION).

9. Indexing Strategy

Goal: Support common queries without over-indexing.

Core indexes:

- products(org_id, sku)
- stock_items(org_id, branch_id, product_id)
- stock_movements(org_id, branch_id, product_id, created_at)
- sales_orders(org_id, branch_id, status, created_at)
- payment_intents(org_id, order_id, status)
- user_memberships(org_id, user_id)

Composite indexes: Add based on query patterns (monitor slow queries).

10. Constraints & Business Rules

- **No negative stock:** Enforced via application logic (check before update); DB check `quantity_on_hand ≥ 0`.
- **Reserved ≤ on_hand:** Check constraint on stock_items.
- **Unique SKU per org:** UNIQUE (org_id, sku) on products.
- **Soft delete:** deleted_at IS NULL filter in repository.
- **Tenant isolation:** All queries include org_id.

11. Data Lifecycle

11.1 Creation

- Validate input (Zod schemas).
- Set `created_by` from auth context.
- Initialize `created_at`, `updated_at`.

11.2 Updates

- Update `updated_at` on every change.
- Audit significant changes (e.g., price, stock adjustments).

11.3 Deletes

- **Soft delete:** Set `deleted_at`; hide from queries.
- **Hard delete:** Retention period (OPEN DECISION: 90 days), then purge.

11.4 Archiving

- Not implemented in MVP; use soft delete.

12. Migrations

Tool: `prisma migrate`.

Process:

1. Dev: `prisma migrate dev --name <description>`.
2. Staging: `prisma migrate deploy`.
3. Prod: `prisma migrate deploy` (after backup).

Rules:

- Never manual schema changes in prod.
- Backward-compatible changes first (add column, deploy code, backfill, switch).
- Destructive changes require explicit approval and rollback plan.

13. Backups & Recovery

- **Backups:** Daily automated (PaaS).
- **Retention:** 7–30 days (configurable).
- **RPO:** < 24 hours.
- **RTO:** < 4 hours.

- **Restore tests:** Quarterly.

14. Security

- **Encryption:** TLS in transit; at rest (PaaS).
- **Access:** Backend only; no direct client access.
- **Secrets:** Env vars; never in code.
- **Audit:** All sensitive actions logged.
- **PII:** Minimize; encrypt if required (future).

15. Performance

- **Connection pooling:** Prisma default; tune if needed.
- **Slow queries:** Monitor; add indexes or denormalize.
- **Large tables:** Partitioning (future) if needed.
- **Caching:** In-memory config; frontend TanStack Query.

16. Observability

- **Logs:** Structured (Pino) with `requestId`, `orgId`, `userId`.
- **Metrics:** Query latency, error rates, slow queries.
- **Tracing:** Correlation IDs across services.

17. Data Governance

- **Ownership:** Org owns all data; user owns personal data.
- **Access:** Role-based; least privilege.
- **Retention:** Audit logs ≥ 5 years (CBN compliance); soft-deleted data 90 days (OPEN DECISION).
- **Export/delete:** NDPA-compliant (future).

18. Open Decisions

ID	Decision	Options	Impact	Recommended Info Needed
DB-01	Hard delete retention	30/90/180 days	Storage, compliance	Legal/NDPA advice
DB-02	Audit log retention	1/5 years	Storage, compliance	CBN/NDPA requirements
DB-03	RLS adoption	App-layer vs RLS	Security, complexity	Scale, threat model
DB-04	Money representation	Integer minor vs DECIMAL	Precision, DX	Accounting requirements

19. AI Coding Agent Rules

AI agents must:

- Read this doc before changing schema.
- Search for existing tables/columns before adding.
- Update migrations and this doc together.
- Never bypass constraints or tenant isolation.
- Add indexes only after measuring slow queries.
- Preserve backward compatibility in migrations.

20. Database Quality Checklist

- ☐ All tables have `org_id` and soft delete.
- ☐ FKs and unique constraints defined.
- ☐ Indexes on common query paths.
- ☐ Money as integer minor units.
- ☐ Timestamps in UTC.
- ☐ Audit logs for sensitive actions.
- ☐ Migrations tested in staging.
- ☐ Backups configured.
- ☐ No direct client access.
- ☐ Tenant isolation enforced.

End of database.md